

Prague University of Economics and Business

Faculty of Informatics and Statistics



ANALYSIS OF FACTORS AFFECTING CLIENT PROFITABILITY

BACHELOR THESIS

Study programme: Data Analytics

Author: Jan Adam Úředníček

Bachelor Thesis Supervisor: Ing. Pavel Zimmermann, Ph.D

Prague, May 2025

Acknowledgement

I would like to thank to Raiffeisenbank a.s. for the data that they have provided for this thesis. Without their data this thesis would not be possible.

I would like to express my gratitude to my thesis advisor, Ing. Pavel Zimmermann, Ph.D., for his invaluable guidance and support throughout writing this thesis.

Abstrakt

Tato práce zkoumá vliv identifikace a kvantifikace klíčových finančních a behaviorálních charakteristik na predikci hrubého zisku (HZ) pro mikropodniky a malé podniky v Raiffeisenbank. Za tímto účelem jsou použity regresní modely strojového učení, které využívají data na úrovni klientů zahrnující různé finanční aktivity, transakční chování a statické atributy, spolu s cílovou proměnnou průměrného hrubého zisku. Analýza se zaměřuje na identifikaci nejvlivnějších charakteristik a hodnocení prediktivní výkonnosti různých modelů. Tři hlavní zjištění jsou, že (i) modely strojového učení mohou efektivně predikovat významnou část variability HZ, (ii) určité finanční a behaviorální charakteristiky klientů, jako jsou průměrné objemy aktiv a pasiv, příjmy a aktivita mezinárodních plateb, se ukazují jako silné pozitivní prediktory HZ, a (iii) pečlivě vybraná podmnožina nejdůležitějších charakteristik může dosáhnout srovnatelné nebo dokonce mírně lepší prediktivní výkonnosti než modely využívající celou sadu charakteristik. Tato práce představuje úvodní datově orientované zkoumání faktorů ovlivňujících hrubý zisk v klientské bázi mikropodniků a malých podniků Raiffeisenbank s využitím moderních technik strojového učení.

Klíčová slova

Predikce hrubého příjmu, Bankovníctví, Strojové učení, XGBoost, Feature Importance

Abstract

This thesis investigates the effects of identifying and quantifying key financial and behavioral features on the prediction of Gross Income (GI) for micro and small segment enterprises at Raiffeisenbank. For this purpose, machine learning regression models are employed, utilizing client-level data on various financial activities, transactional behavior, and static attributes, along with the target variable of average gross income. The analysis focuses on identifying the most influential features and evaluating the predictive performance of different models. The three main findings are that (i) machine learning models can effectively predict a significant portion of the variability in GI, (ii) certain client financial and behavioral features, such as average asset and liability volumes, income, and international payment activity, emerge as strong positive predictors of GI, and (iii) a carefully selected subset of the most important features can achieve comparable or even slightly better predictive performance than models utilizing the entire feature set. This thesis represents an initial data-driven investigation into the drivers of Gross Income within Raiffeisenbank's micro and small enterprise client base using modern machine learning techniques.

Keywords

Gross Income Prediction, Banking, Machine Learning, XGBoost, Feature Importance

Contents

Business case	7
Introduction.....	8
1 Machine Learning	10
1.1 Distinction Between Black-Box and Interpretable (Mathematical) Models	10
1.2 Decision trees	11
1.3 Ensemble learning.....	11
1.4 Boosting.....	12
1.5 XGBoost.....	13
1.6 Feature engineering	14
1.7 Feature importance	15
1.8 SHAP	16
1.9 Model Validation	16
1.9.1 Hyperparameter tuning	17
1.10 Model Performance Evaluation	18
1.10.1 Mean Absolute Error (MAE)	18
1.10.2 Root Mean Square Error (RMSE).....	18
1.10.3 R-squared (Coefficient of Determination)	19
2 Data and Feature engineering	21
2.1 Dataset.....	21
2.1.1 General information attributes	21
2.1.2 Product attributes	23
2.1.3 Financial behaviour attributes.....	24
2.2 Feature engineering	26
2.3 Handling of missing values.....	26
3 Modeling.....	28
3.1 Splitting the dataset.....	28
3.2 Getting baseline model	28
3.3 Choosing model	29
3.4 XGBoost model	29
3.5 Feature selection	30
4 Comparison of models	32
4.1 Feature comparison.....	33
4.2 Limitation of the results.....	35

5 Recommendation and insights	36
Conclusions.....	39
List of references	40
Appendices	II
Appendix A: Source code	II

Business case

The primary business objective of this research is to identify and quantify the key financial and behavioral features that significantly influence prediction of Gross Income (GI) within the bank's client base. Understanding these drivers is crucial for strategic decision-making, enabling the bank to prioritize areas of focus for maximizing GI and optimizing client relationships. This study aims to provide actionable insights into the factors that contribute most to the bank's profitability.

However, while Gross Income is a readily calculable metric for each client, its value is the result of a complex interplay of numerous financial and behavioral factors that can vary significantly at an individual level. Therefore, this research aims to move beyond a simple accounting of GI to identify the core drivers that can be reliably used for prediction and strategic understanding.

Specifically, this research seeks to answer the central question:

What are the most important financial and behavioral characteristics of our clients that drive predicted Gross Income, and how can we leverage this understanding to inform strategic decisions within the bank?

Evaluation Framework (to be used later in the thesis):

The success of this research will be evaluated based on the predictive accuracy of the developed models, the stability and quantified impact of the identified key features, and the practical relevance of the resulting strategic insights for the bank.

Introduction

The banking sector's financial performance is significantly driven by the Gross Income (GI) generated from its client base. Understanding the factors that influence this income is crucial for informed strategic decision-making, allowing banks to optimize resource allocation and enhance client relationships. The increasing availability of detailed client financial and behavioral data presents an opportunity to employ advanced analytical techniques to gain deeper insights into the determinants of GI.

Traditional methods of assessing client value and income contribution can be limited. Leveraging machine learning offers the potential to identify specific client characteristics and behaviors that have the most significant impact on the prediction of GI. This data-driven approach can provide a more nuanced understanding than conventional methods, enabling banks to refine their strategies and focus on key drivers of profitability.

This bachelor's thesis investigates the relationship between client-level financial and behavioral features and the resulting Gross Income generated by Raiffeisenbank a.s. While GI is a calculable metric for each client, its value is influenced by a multitude of underlying factors that can vary significantly at an individual client level. Therefore, the primary objective of this research extends beyond simple calculation to develop robust predictive models that identify and quantify the key factors influencing GI. To achieve this, the study primarily employs a powerful ensemble machine learning technique, specifically boosting algorithms (XGBoost), which are adept at capturing complex non-linear patterns and often yield high predictive accuracy. While an initial linear regression model was explored, its limited performance led to a focus on the more sophisticated capabilities of boosting for the primary analysis. The interpretability of the boosting model, primarily through the analysis of feature importance using SHAP values, is central to deriving actionable insights for the bank's strategic planning.

The research seeks to answer the following questions: First, which financial attributes (e.g., deposit volumes, loan portfolios) exhibit the strongest positive correlation with higher GI, as identified by the boosting model? Second, what behavioral patterns (e.g., transaction frequency, product usage, international payment activity) are indicative of greater GI contribution? Third, can the boosting model effectively predict GI and identify the relative importance and impact (positive or negative) of the contributing features, as revealed by SHAP analysis?

To address these questions, this study utilizes a comprehensive dataset of client financial and behavioral data from Raiffeisenbank a.s.. The primary analysis focuses on the implementation and evaluation of the XGBoost model, with SHAP value analysis being a key output for feature interpretation. The comparative evaluation against baseline models, including initial linear regression attempts, provides insights into the suitability of different approaches for this specific application.

This thesis is structured into five chapters. Chapter 1 reviews relevant literature on applied machine learning, with a focus on boosting techniques and feature importance interpretation using SHAP. Chapter 2 details the data used, the feature engineering process, and the data anonymization applied for security. Chapter 3 outlines the experiment design and modeling procedures, emphasizing the XGBoost model and the iterative feature selection based on SHAP values. Chapter 4 presents the results of the XGBoost model's performance and the detailed analysis of feature importance and impact using SHAP. Chapter 5 concludes the thesis with a summary of the findings and strategic recommendations for the bank.

Disclaimer: The data used in this thesis cannot be published as the data belongs to the Raiffeisenbank a.s..

1 Machine Learning

1.1 Distinction Between Black-Box and Interpretable (Mathematical) Models

"Too much of a black box to me" is a long-standing criticism of data-driven machine learning methods, particularly (deep) neural networks (Escalante et al., Eds., 2019). Despite this, these black boxes have achieved astounding results, arguably because researchers no longer aim for full control over model internals, allowing the data to optimize the system end-to-end for superior performance (Escalante et al., Eds., 2019).

However, the complexity of modern models, often with millions of parameters, makes it hard to understand their internal processes (Escalante et al., Eds., 2019). This raises concerns about trust, especially in safety-critical applications (Escalante et al., Eds., 2019). Furthermore, these black boxes can struggle to generalize, exhibit overconfidence in wrong predictions for unseen data, and are vulnerable to adversarial examples (Escalante et al., Eds., 2019). As AI performance reaches or surpasses human levels, the black-box criticism becomes even more relevant (Escalante et al., Eds., 2019).

While black-box models prioritize accuracy through intricate, often opaque functions, interpretable models aim for transparency, allowing humans to understand the relationship between inputs and outputs. As Rudin (2019) argues in her abstract, the practice of trying to explain black box models, rather than creating models that are interpretable in the first place, is problematic and can perpetuate bad practices.

The choice between accuracy and interpretability often presents a trade-off (Rudin, 2019). While complex black boxes might capture intricate patterns for higher accuracy, interpretable models offer insights into the "why" behind predictions. In high-stakes domains like finance, understanding the reasoning is crucial for trust and accountability (Rudin, 2019).

Despite the power of black boxes, the need for explainability and interpretability is growing. As highlighted in *Explainable and Interpretable Models in Computer Vision and Machine Learning* (Escalante et al., Escalera, Guyon, Baró, Güçlütürk, Güçlü, & van Gerven, Eds., 2019), the next generation of machine learning should not only predict accurately but also explain its results and allow for model interpretation. This thesis, while leveraging a powerful predictive model, recognizes the importance of understanding the drivers of Gross Income at Raiffeisenbank, aligning with the call for more interpretable approaches, especially in domains with significant real-world impact (Rudin, 2019).

1.2 Decision trees

Decision trees are prediction models constructed by recursively partitioning a dataset and fitting a simple model to each partition (Loh, 2011). Their name comes from the usual practice of describing the partitioning process by a decision tree (Smith, 2018). This structure visually represents a step-by-step process for classification or regression.

A Decision Tree is simply a step-by-step process to go through to decide a category something belongs to. This involves a flow chart of questions to determine what type of "something" it is (Smith, 2018). For classification trees, designed for dependent variables that take a finite number of unordered values, the partitioning aims to minimize misclassification cost (Loh, 2011). For regression trees, used for dependent variables that take continuous or ordered discrete values, the partitioning typically minimizes the squared difference between observed and predicted values (Loh, 2011).

To use the decision tree, you start at the top and begin hitting each decision in series. At each point, you need to make a choice on which way to go between exactly two options. Eventually you reach the bottom and have a decision as to the outcome (Smith, 2018). The partitioning process can be represented graphically as a decision tree (Smith, 2018, Loh, 2011).

1.3 Ensemble learning

Ensemble learning is a paradigm in machine learning that deviates from the traditional approach of striving for a single, highly accurate model (Burkov, 2019). Instead, it centers on training a multitude of low-accuracy models, often referred to as "weak learners," and subsequently aggregating their individual predictions to construct a high-accuracy "meta-model" (Burkov, 2019).

The rationale behind this approach lies in the principle that while individual weak learners may possess limited predictive capability, their collective wisdom, when appropriately combined, can yield a robust and accurate overall prediction. These low-accuracy models are typically generated by learning algorithms constrained in their complexity, resulting in fast training and prediction times (Burkov, 2019).

A frequently employed weak learner in ensemble methods is a decision tree learning algorithm where the tree growth is intentionally limited, often by stopping the splitting process after only a few iterations (Burkov, 2019). This results in shallow, less accurate individual trees. However, the core idea of ensemble learning is that if these individual trees are diverse (i.e., not identical) and each performs at least slightly better than random

guessing, then combining a large number of such trees through a voting mechanism can lead to a highly accurate meta-model (Burkov, 2019).

To generate a prediction for a given input x , the predictions from each of the constituent weak models are combined using a form of weighted voting (Burkov, 2019). The specific method of weighting votes can vary depending on the ensemble algorithm employed. However, the underlying principle remains consistent: if a consensus emerges among the ensemble of weak models towards a particular prediction (e.g., classifying an email as "spam"), then that label is assigned to the input x (Burkov, 2019).

Two of the most widely adopted and effective ensemble learning algorithms are Random Forest and Gradient Boosting (Burkov, 2019). These methods leverage the power of combining numerous weak learners, typically shallow decision trees, in distinct ways to achieve state-of-the-art predictive performance in a wide range of machine learning tasks.

1.4 Boosting

Another highly effective ensemble learning algorithm is gradient boosting (Burkov, 2019). Unlike methods that train weak learners independently, gradient boosting adopts a sequential approach, where each new model attempts to correct the errors made by the previous ones (Burkov, 2019).

Let's first consider gradient boosting for regression. The process begins by establishing a simple, constant model, often the average of the target variable in the training data (Burkov, 2019). Following this initialization, the algorithm iteratively refines the model. In each iteration, the "labels" of the training examples are modified to represent the **residuals**, which are the differences between the actual target values and the predictions made by the current ensemble model (Burkov, 2019).

A new, weak decision tree model is then trained using this modified training set, where the residuals serve as the target variable instead of the original labels (Burkov, 2019). This new tree essentially learns to predict the errors of the existing ensemble. The boosting model is then updated by adding this newly trained tree, scaled by a **learning rate** (η), which is a crucial hyperparameter that controls the contribution of each new tree to the overall model (Burkov, 2019). This step can be represented as:

$$f = f_{previous} + \eta * f_{new\ tree} \quad (2.1)$$

This process of calculating residuals, training a new tree to predict these residuals, and adding the tree to the ensemble (with the learning rate) is repeated for a predefined number of iterations, up to a maximum number of trees (M), which is another important hyperparameter (Burkov, 2019). Intuitively, by focusing on the residuals, each subsequent tree aims to reduce the prediction errors of the preceding trees, gradually building a strong regressor (Burkov, 2019).

The term "gradient" in gradient boosting arises from its connection to gradient descent principles, although it doesn't directly calculate gradients in the same way as in linear regression (Burkov, 2019). In linear regression, the gradient of the cost function guides the adjustment of parameters to minimize the error. In gradient boosting, the residuals act as a proxy for this gradient, indicating the direction in which the model needs to be adjusted to reduce the error (Burkov, 2019). The learning rate then controls the step size in this "gradient" direction, similar to its role in gradient descent.

For classification tasks, the gradient boosting algorithm follows a similar sequential logic, but with slight modifications tailored for classification problems, particularly binary classification (Burkov, 2019). The prediction of the ensemble of regression trees is often transformed into a probability using a sigmoid function, similar to logistic regression. The algorithm then aims to maximize the likelihood of the observed labels. In each iteration, it calculates the partial derivatives (akin to gradients) of the loss function with respect to the current model's predictions. These derivatives are then used as the "target" to train a new regression decision tree. An optimal update step is determined, and the new tree is added to the ensemble, again controlled by a learning rate (Burkov, 2019).

Gradient boosting is recognized as one of the most powerful machine learning algorithms, capable of achieving high accuracy and handling large datasets with numerous examples and features (Burkov, 2019). While it often outperforms Random Forest in terms of accuracy, its sequential nature can lead to longer training times compared to the parallel training of Random Forest (Burkov, 2019). The key hyperparameters to tune in gradient boosting include the number of trees, the learning rate, and the depth of the individual trees, all of which significantly impact the model's accuracy and training/prediction speed (Burkov, 2019). Notably, training on residuals effectively optimizes the overall model for the mean squared error criterion in regression, and a similar principle applies to the loss function in classification. Unlike bagging methods like Random Forest that primarily aim to reduce variance, boosting algorithms like gradient boosting focus on reducing bias (underfitting), although careful tuning is essential to avoid overfitting (Burkov, 2019).

1.5 XGBoost

Building upon the foundational principles of gradient boosting (Burkov, 2019), a particularly influential and high-performing implementation of this ensemble technique is XGBoost (Chen & Guestrin, 2016). While gradient boosting, as described previously, iteratively refines a model by sequentially training weak learners to correct the errors of their predecessors (Burkov, 2019), XGBoost introduces significant algorithmic and system-level optimizations that contribute to its enhanced scalability and performance across a wide array of machine learning tasks (Chen & Guestrin, 2016).

XGBoost distinguishes itself through several key innovations. One notable aspect is its sophisticated handling of the loss function. Beyond the traditional mean squared error for regression or likelihood maximization for classification, XGBoost incorporates a regularized objective function. This objective includes both a loss function term and a regularization term that penalizes the complexity of the individual trees within the ensemble (Chen &

Guestrin, 2016). This regularization helps to prevent overfitting, leading to more generalizable models (Chen & Guestrin, 2016).

Furthermore, XGBoost employs a novel approach to finding the best splits in the tree-building process. While basic gradient boosting might use a greedy approach, XGBoost offers both an exact greedy algorithm for smaller datasets and an approximate algorithm that can efficiently handle large-scale data (Chen & Guestrin, 2016). This approximate algorithm utilizes a weighted quantile sketch to efficiently identify potential split points, even when dealing with massive datasets and instance weights (Chen & Guestrin, 2016).

A significant advantage of XGBoost lies in its inherent scalability. The system is engineered for parallel computation, allowing it to leverage multi-core processors and distributed computing environments to significantly accelerate training times (Chen & Guestrin, 2016). Moreover, XGBoost incorporates a sparsity-aware algorithm, which efficiently handles datasets with missing values or a high proportion of zeros, a common characteristic of real-world data (Chen & Guestrin, 2016). This optimization avoids unnecessary computations on zero or missing feature values, further enhancing performance.

Beyond algorithmic enhancements, XGBoost includes system-level optimizations such as cache-aware data structures and out-of-core computation. The cache-aware design optimizes data access patterns during training, leading to better utilization of hardware resources and faster processing (Chen & Guestrin, 2016). Out-of-core computation enables XGBoost to handle datasets that are too large to fit into memory by efficiently processing data in chunks from disk (Chen & Guestrin, 2016).

The effectiveness and efficiency of XGBoost have made it a dominant algorithm in various machine learning competitions and real-world applications, often outperforming standard gradient boosting implementations due to its advanced regularization techniques, efficient split finding algorithms, and robust system optimizations for scalability and speed (Chen & Guestrin, 2016). Its ability to handle large and sparse datasets efficiently, coupled with its regularization strategies, contributes to its reputation as a state-of-the-art tree boosting system (Chen & Guestrin, 2016).

1.6 Feature engineering

Effective feature engineering is paramount for building accurate classification and prediction models. While feature selection techniques exist, the process of creating good features with high predictive power is a crucial prerequisite for achieving optimal performance (Rawat & Khemchandani, 2019).

Feature engineering involves transforming raw data into a set of features that can be effectively understood and utilized by a learning algorithm. These features serve as the fundamental representation of the data for the model (Rawat & Khemchandani, 2019). The quality of these features directly impacts the accuracy and effectiveness of the resulting classification or prediction system. Poorly engineered features that do not adequately

capture the underlying problem can hinder the learning process and lead to suboptimal results (Rawat & Khemchandani, 2019).

Feature engineering is often considered a challenging and time-consuming aspect of developing successful models, requiring domain expertise and a deep understanding of the data (Rawat & Khemchandani, 2019). The goal is to create features that are strongly relevant to the prediction task, potentially combining weakly relevant features to enhance their predictive power, while discarding irrelevant and redundant information. This process occurs after initial data preparation and cleaning, but before the model training phase, aiming to provide the learning algorithm with the best possible data representation (Rawat & Khemchandani, 2019).

1.7 Feature importance

In many real-world datasets, particularly those used for complex prediction tasks, the number of features (attributes) can be substantial. When dealing with such high-dimensional data, it becomes crucial to understand the relative importance of each feature in the model's predictions (Hartshorn, 2023). Identifying which features are most influential allows analysts to focus their attention and gain deeper insights into the underlying relationships within the data (Hartshorn, 2023).

When a model is trained on a dataset with numerous features, it learns to utilize these features to make predictions. However, not all features contribute equally to the model's decision-making process. Some features might have a strong impact on the predicted outcome, while others might be less informative or even redundant (Hartshorn, 2023). Quantifying feature importance provides a way to understand this differential contribution.

Various methods exist across different machine learning algorithms to determine feature importance. Generally, these methods aim to assess the degree to which a model's performance changes when a particular feature is either removed, perturbed, or considered in isolation. Features that cause a significant drop in performance when altered or are heavily weighted in the model's structure are typically deemed more important (Hartshorn, 2023).

Understanding feature importance offers several benefits. It helps in understanding which variables the model relies on most heavily to make predictions, providing transparency into the model's behavior (Hartshorn, 2023). It can guide feature selection processes by identifying less important features that might be removed to simplify the model, reduce computational cost, and potentially improve generalization (Hartshorn, 2023). The importance assigned to different features can provide valuable insights into the underlying domain and the factors that truly drive the outcome being predicted (Hartshorn, 2023). Furthermore, unexpectedly low or high importance of certain features can highlight potential issues with data quality or feature engineering (Hartshorn, 2023).

While the specific mechanisms for calculating feature importance vary depending on the model type, the underlying goal remains the same: to quantify the contribution of each input

variable to the model's predictive power. This understanding is essential for building trust in the model, gaining actionable insights from it, and potentially improving its design and performance (Hartshorn, 2023).

1.8 SHAP

Understanding why a model makes a prediction is crucial, but complex models often lack interpretability (Lundberg & Lee, 2017). SHAP (SHapley Additive exPlanations) offers a unified framework to address this by assigning an importance value to each feature for a specific prediction (Lundberg & Lee, 2017).

SHAP views any explanation as a model itself and defines a class of additive feature attribution methods, unifying several existing interpretation techniques (Lundberg & Lee, 2017). It leverages game theory to show that SHAP values are the unique additive measure satisfying desirable properties (Lundberg & Lee, 2017).

SHAP values represent the contribution of each feature to the difference between the actual prediction and the expected prediction, considering the effect of conditioning on that feature (Lundberg & Lee, 2017). For non-linear models or dependent features, SHAP values average contributions across all possible feature orderings.

While exact computation can be hard, SHAP provides model-agnostic (e.g., Kernel SHAP) and model-specific (e.g., Tree SHAP) approximation methods (Lundberg & Lee, 2017). Kernel SHAP, for instance, connects local linear explanations with Shapley values through weighted regression. By providing a consistent and theoretically sound way to quantify feature importance, SHAP helps build trust, understand model behavior, and potentially improve model design (Lundberg & Lee, 2017).

1.9 Model Validation

To ensure machine learning models perform well on new, unseen data, datasets are typically split into three parts: a training set, a validation set, and a test set (Burkov, 2019). The training set builds the model, while the validation and test sets are "hold-out" sets the model doesn't learn from (Burkov, 2019). Common splits range from 70/15/15 to 95/2.5/2.5 for large datasets.

The purpose of hold-out sets is to evaluate how well the model generalizes beyond the training data (Burkov, 2019). The validation set is used to select the best learning algorithm and fine-tune its hyperparameters (Burkov, 2019). The test set provides a final, unbiased assessment of the model's performance before deployment (Burkov, 2019).

When a separate validation set is impractical, **cross-validation** can be used. The training data is split into several "folds." The model is trained multiple times, each time using a different fold as a temporary validation set. The average performance across these folds estimates how well the model might generalize (Burkov, 2019). Grid search with cross-validation helps find optimal hyperparameters. The final model, trained on the entire training set with the best hyperparameters, is then evaluated on the test set for a realistic performance estimate (Burkov, 2019). This rigorous validation process ensures the model is not just memorizing training data but can effectively predict outcomes for new, unseen instances.

1.9.1 Hyperparameter tuning

Hyperparameter tuning is the process of selecting the optimal values for an algorithm's settings that are not learned from the data itself (Burkov, 2019). These settings, like the learning rate or regularization strength, significantly impact a model's performance, and finding the right combination is crucial (Burkov, 2019).

A fundamental technique is **grid search**, where a data analyst defines a specific set of potential values for each hyperparameter. The algorithm then systematically trains and evaluates a model for every possible combination of these values, using a validation set to assess performance (Burkov, 2019). The hyperparameter set that yields the best results on the validation data is chosen as optimal. While straightforward, grid search can be computationally expensive, especially with many hyperparameters or a wide range of potential values (Burkov, 2019).

More efficient alternatives exist. **Random search** involves defining a probability distribution for each hyperparameter and then randomly sampling a fixed number of combinations to evaluate. This can often find good hyperparameter settings more quickly than grid search, especially in high-dimensional spaces where not all hyperparameters are equally important (Burkov, 2019). **Bayesian optimization** takes a more informed approach. It builds a probabilistic model of the objective function (the performance metric on the validation set) and uses this model to intelligently suggest the next set of hyperparameters to evaluate, aiming to efficiently find the optimum by balancing exploration and exploitation of the hyperparameter space (Burkov, 2019).

Once the best hyperparameters are identified using a validation set (or through cross-validation when data is limited), the final model is trained on the entire training dataset using these optimized settings. Finally, the model's true generalization ability is assessed on a completely separate test set, providing an unbiased estimate of its performance on unseen data (Burkov, 2019). Modern machine learning libraries and specialized tools offer a range of hyperparameter tuning methods, simplifying and automating this critical step in model development (Burkov, 2019).

1.10 Model Performance Evaluation

Evaluating the performance of a regression model is a crucial step in the machine learning workflow. For models predicting continuous numerical values, several metrics help assess their accuracy. This section will discuss three commonly used metrics: Mean Absolute Error (MAE), Root Mean Square Error (RMSE), and R-squared.

1.10.1 Mean Absolute Error (MAE)

Mean Absolute Error (MAE) is a straightforward metric that measures the average absolute difference between the predicted values and the actual target values (Ahmed, 2023). Unlike metrics that square the errors, MAE treats all errors equally, regardless of whether the model overestimates or underestimates.

The formula for MAE is:

$$MAE = \frac{1}{n} * \sum |y_i - \hat{y}_i| \quad (2.2)$$

Where (n) is the number of data points, (y_i) is the actual target value for data point (i), and ($\hat{y}_i$) is the predicted value for data point (i).

MAE is valuable because it is less sensitive to extreme values in the data compared to metrics like Mean Squared Error (MSE) or RMSE. Additionally, MAE is expressed in the same units as the target variable, making it easy to understand the average magnitude of the prediction errors. For example, if predicting Gross Income in a specific currency, the MAE will also be in that currency. Its simplicity in calculation and interpretation provides a clear sense of the model's overall performance in terms of average absolute deviations. A lower MAE indicates that the model's predictions are, on average, closer to the actual values. When interpreting the MAE, it's essential to consider the scale of the target variable; an MAE that might be acceptable for a large-scale prediction could be unacceptable for a prediction on a smaller scale.

1.10.2 Root Mean Square Error (RMSE)

The Root Mean Square Error (RMSE) is another commonly used metric that measures the prediction errors of a regression model (Jacobi, 2023). It represents the square root of the mean of the squared differences between the predicted and actual values. In simpler terms, it first calculates the difference between each predicted value and its corresponding actual value. Then, each of these differences (also known as residuals) is squared. Next, the average of these squared differences is computed, and finally, the square root of this average is taken to obtain the RMSE.

RMSE gives more weight to larger errors due to the squaring operation before averaging, making it more sensitive to outliers than MAE (Jacobi, 2023). Consequently, RMSE

provides a more conservative estimate of model accuracy when large errors are particularly undesirable. Similar to MAE, RMSE is also in the same unit as the target variable, aiding in interpretability. A lower RMSE indicates a better fit of the model to the data. In practice, RMSE is often used to compare the performance of different regression models, with the model exhibiting the lowest RMSE generally considered superior, although a comprehensive evaluation should include other metrics as well. RMSE formula Where (n) is the number of data points, (y_i) is the actual target value for data point (i) and ($\hat{y}_i$) is the predicted value for data point (i)

$$RMSE = \sqrt{\frac{1}{n} * \sum (y_i - \hat{y}_i)^2} \quad (2.3)$$

Distinguishing RMSE from MAE: While both metrics quantify the average prediction error, RMSE penalizes larger errors more heavily than MAE. If the dataset contains significant outliers, RMSE will typically be higher than MAE. The choice between them depends on the specific application and the importance of penalizing large errors (Jacobi, 2023).

1.10.3 R-squared (Coefficient of Determination)

R-squared (R^2), also known as the coefficient of determination, is a statistical measure that indicates the proportion of the variance in the dependent variable (the target) that is predictable from the independent variables (the features) in a regression model (Liu, 2023). It ranges from 0 to 1, where 1 signifies that the model perfectly explains all the variance in the target variable, and 0 suggests that the independent variables explain none of the variance.

The formula for R-squared is:

$$R^2 = 1 - \frac{\text{Unexplained Variation}}{\text{Explained Variation}} \quad (2.4)$$

Where Unexplained Variation is the sum of the squared differences between the actual values and the predicted values, and Total Variation is the sum of the squared differences between the actual values and the mean of the actual values.

R-squared essentially tells you how well the model fits the observed data (Liu, 2023). For instance, an R-squared of 0.70 implies that 70% of the variance in the target variable can be explained by the model's features. However, R-squared has limitations. In multiple regression models, the R-squared value can increase simply by adding more variables, even if those variables do not genuinely improve the model's predictive power. Adjusted R-squared addresses this by penalizing the addition of unnecessary predictors. Furthermore, a high R-squared does not necessarily mean the model is good, nor does a low R-squared mean it is bad. R-squared measures the linear relationship between variables and might not

capture complex, non-linear relationships effectively. It also doesn't indicate if the model is biased.

In the context of evaluating the Gross Income prediction models in this thesis, MAE, RMSE, and R-squared provide valuable insights. MAE and RMSE offer an understanding of the average prediction error in the units of Gross Income, with RMSE highlighting the impact of larger errors. R-squared indicates the proportion of the variance in GI that the model can explain based on the financial and behavioral features. Together, these metrics help assess the accuracy and the goodness of fit of the developed models.

2 Data and Feature engineering

2.1 Dataset

Raiffeisenbank provided the dataset for this analysis, which encompasses 100,000 randomly selected micro and small segment enterprises. This dataset includes a variety of client information, such as behavioral data, details sourced from the commercial register, and transaction history. The data was extracted on March 31st, 2025, meaning that many of the columns reflect client behavior up to that point, particularly within that month. For security purposes and to avoid disclosing sensitive bank information, each non-count numeric column in the dataset has been multiplied by a unique random number ranging between 0.7 and 1.7.

The target variable for our predictive modeling is `AVG_GROSS_INCOME_6M_LCY`. This represents the client's average monthly gross income in czech crowns over the preceding six months. For newer clients who haven't been with the bank for a full six months, the average is calculated based on their tenure with the bank.

It's important to note that while labeled 'Gross Income,' this variable already incorporates certain deducted costs, such as fees from Apple and Google payment systems. Consequently, the value of `AVG_GROSS_INCOME_6M_LCY` can, and frequently does, result in negative figures.

2.1.1 General information attributes

Table 2.1

Description of General information attributes in the dataset (Author's calculation)

COLUMN	VALUE TYPE	DESCRIPTION
<code>PARTYTYPE_CODE</code>	Bool	Whether client is a legal entity or an entrepreneur
<code>GENDER_CD</code>	String	Gender
<code>MARITAL_STATUS_CD</code>	String	Married, Single, Widow

CUST_CARE	String	Type of care customer is in
NUM_EMPLOYEES_CNT	Integer	Number of employees
RESIDENCE_STATUS_CD	String	CZ, EU or other
COUNTRY_OF_RESIDENCE_CD	String	CZ, EU or other
PRIMARY_CITIZENSHIP_COUNTRY_CD	String	CZ, EU or other
PERSONAL_BANKER_FLG	Bool	Whether client has a personal banker or not
MORTGAGE_SERVICE_FLG	Bool	Has a mortgage service
INVESTMENT_SRVC_SUBSCRIBED_FLG	Bool	Subscribed to an investment service
INVESTMENT_EXPERIENCE_YRS_NO	Integer	Years of investment experience
CREDIT_CONTRACT_INSURANCE_FLG	Bool	Insurance against default
COUNTRY_CONTACT_CD	String	CZ, EU or other
LEGALFORM_CODE	String	Code of legal entity
AGE	Integer	Age

BANKER_PRIMARY_PORTFOLIO	Bool	Part of Bankers primary portfolio
TRANSACTION_SCORE	Float	Transaction score
TRANS_URG_GRADE	Integer	Urgency of transactions
SPB_FLG	Bool	Has mobile banking
GCG_FLG	Bool	member of an economically related group
PARTY_ACTIVE_BANKING_FLG	Bool	Client actively uses his account
NACE_RIZIKO	Bool	Whether client does business in risky sector
EXEKUCE_FLG	Bool	Whether client has a foreclosure or not
CORP_CARE_FLG	Bool	Corporate care
COMP_FOUNDATION_DATE	Date	Date when business was founded
MULTICHANNEL_FLG	Bool	Client exists in multichannel
CONTACT_ADDR_HOME_BRANCH_DIST	Float	Distance from home address to closest branch
CUST_START_DATE	Date	Date when party became client
ASCH_KO_MICRO_INVU	String	Whether client has investment loan or has an offer
ASCH_KO_MICRO_KTK	String	Whether client has overdraft or has an offer

2.1.2 Product attributes

Table 2.2

Description of Product attributes in the dataset (Author's calculation)

Column	Value type	Description
TOTAL_LIVE_CNT	Integer	Total active products
CA_TOTAL_CNT	Integer	Total active customer accounts
SA_TOTAL_CNT	Integer	Total active saving accounts
TD_TOTAL_CNT	Integer	Total active term deposits
INV_TOTAL_CNT	Integer	Total active investing accounts
OVD_TOTAL_CNT	Integer	Total active overdrafts

CC_TOTAL_CNT	Integer	Total active credit cards
LOAN_TOTAL_CNT	Integer	Total number of loans
MORTG_TOTAL_CNT	Integer	Total number of mortgages
GARANACE_TOTAL_CNT	Integer	Total number of guarantees
DOCPAY_TOTAL_CNT	Integer	Total number of docpays
FACT_TOTAL_CNT	Integer	Total number of facts about client
MARG_TR_TOTAL_CNT	Integer	Total number of marginal transactions
TREA_TOTAL_CNT	Integer	Total number of treatments
OTHER_TOTAL_CNT	Integer	Other total count

2.1.3 Financial behaviour attributes

Table 2.3

Description of Financial behaviour attributes in the dataset (Author's calculation)

Column	Value type	Description
GFPTM_CR_TURNOVER_LCY	Float	Summary amount of credit client transaction except intra-client transactions
GFPTM_DB_TURNOVER_LCY	Float	Summary amount of debit client transaction except intra-client transactions
GFPTM_CR_TRANS_CNT	Integer	Number of customer originated credit transaction except intra-client transactions
GFPTM_DB_TRANS_CNT	Integer	Number of customer originated debit transaction except intra-client transactions
GFPTM_CCRD_USAGE_CNT	Integer	Credit card usage count

GFPTM_CCRD_USAGE_LCY	Float	Credit card debit transaction amount in CZK
GFPTM_DCRD_USAGE_CNT	Integer	Debit card usage count
GFPTM_DCRD_USAGE_LCY	Float	Debit card debit transaction amount in CZK
GFPTM_FOREIGN_PAYMENT_IN_CNT	Integer	Incoming foreign payment count
GFPTM_FOREIGN_PAYMENT_IN_LCY	Float	Incoming foreign payment amount
GFPTM_FOREIGN_PAYMENT_OUT_CNT	Integer	Outgoing foreign payment count
GFPTM_FOREIGN_PAYMENT_OUT_LCY	Float	Outgoing foreign payment amount in CZK
GFPTM_CASH_DEPOSIT_CNT	Integer	Cash deposit count
GFPTM_CASH_DEPOSIT_LCY	Float	Cash deposit amount in CZK
GFPTM_CASH_WITHDRAWAL_CNT	Integer	Cash withdrawal count
GFPTM_CASH_WITHDRAWAL_LCY	Float	Cash withdrawal count amount in CZK
GFPPM_AVG_LIA_VOL_LCY	Float	Average volume of liabilities in CZK
GFPPM_AVG_ASS_VOL_LCY	Float	Average volume of assets in CZK
SPB_Login_cnt	Integer	Count of distinct days where client logged into APP
CUST_START_DATE	Date	Date when party became client
AVG_GROSS_INCOME_6M_LCY	Float	Average gross income for 6 months in CZK
INCOME	Float	Income
AVG_CR_3M_AMT	Float	Amount of external credit transactions

2.2 Feature engineering

To enable the machine learning models to understand time-based information, the "CUST_START_DATE" and "COMP_FOUNDATION_DATE" columns were transformed into a numerical format. We calculated the difference in days between these dates and a fixed point in time (March 31st, 2025, the data extraction date). This allows the model to understand how long a client has been with the bank and the age of their company.

When one of the date columns was missing for a client, we used the value from the other available date column. While not perfect, this prevents the model from encountering illogical imputed values, for example, a very young company being a long-term customer.

We also engineered a new feature called "TIME_TO_CUSTOMER_DAYS." This represents the number of days that passed between a company's registration date and the day they became a bank customer. This could help the model learn about the speed of client acquisition.

To track how active clients are, we created three specific metrics: "15_3_activity," "10_10_activity," and "15_10_5_activity." The "15_3_activity" feature marks clients with at least CZK 15,000 in turnover and at least 3 payments as active. The "10_10_activity" feature identifies clients who logged into mobile banking on at least 10 different days and made at least 10 payments. Finally, "15_10_5_activity" combines a turnover and payment requirement with the mobile banking login frequency.

The goal of these new features is to provide the machine learning models with more meaningful information. Representing dates numerically allows the model to understand the time-related aspects of clients and their companies. "TIME_TO_CUSTOMER_DAYS" can show how quickly clients become customers. The activity metrics offer a more detailed way to see how engaged clients are, potentially indicating behaviors linked to higher Gross Income. By calculating these activity levels based on turnover, payments, and logins, we allow the model to capture how effectively clients utilize the bank's services. For simplicity, we did not include features that aimed to capture trends in client behavior over time.

2.3 Handling of missing values

Missing values were addressed using different strategies based on the nature of the features. For "AGE", "CUST_START_DATE", and "COMP_FOUNDATION_DATE", median imputation was chosen. Filling these with 0 would be illogical, as "CUST_START_DATE" and "COMP_FOUNDATION_DATE" were derived from date differences, where 0 would imply a brand-new client or company at the exact time of data collection. The median provides a robust central value less sensitive to outliers.

For the remaining numerical features, missing values were filled with 0. The rationale here is that the absence of data for these features likely indicates the client did not possess that particular product or engage in that specific activity. Treating these missing values as 0

allows the model to potentially differentiate between a true zero value and the absence of the feature.

Both imputation strategies were applied separately to the training and testing sets to prevent data leakage.

3 Modeling

3.1 Splitting the dataset

The initial dataset comprised 100,000 entries. However, 1,140 rows contained missing values in the target variable (Gross Income). These missing values are likely attributable to new clients for whom Gross Income had not yet been calculated. To maintain data integrity and given the sufficient remaining sample size, these rows were removed. Consequently, the dataset was split into training and testing sets using the `train_test_split` function from the `scikit-learn` library, with a ratio of 80% for training and 20% for testing and a random seed of 42 to ensure reproducibility. This resulted in a training set of 79,088 rows and a test set of 19,722 rows.

3.2 Getting baseline model

Initially, I employed linear regression to establish a baseline understanding of the predictive potential for gross income. However, standard linear regression struggled with this dataset. Dummy encoding of categorical features expanded the number of columns beyond 100, leading to significant overfitting. This resulted in poor model performance, characterized by an R^2 value substantially below zero and high Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE).

Subsequently, I investigated more robust linear models, including Ridge Regression (L2 regularization), Lasso Regression (L1 regularization), and Elastic Net (a combination of both). From these models only Elastic Net offered marginal improvements over basic linear regression, but its overall predictive capability remained inadequate.

A notable challenge was the non-normal distribution of the target variable, which included negative values, making straightforward transformations difficult. Attempts to apply a log transformation with a shift to accommodate negative values unfortunately worsened the model's performance.

Table 3.1

Performance of models on training and testing data (Author's calculation)

Model	R^2 Test	MAE Test	RMSE Test	R^2 Train	MAE Train	RMSE Train
OLS	-0.36	1249.07	7258.96	0.63	1237.96	6569.89
Ridge	-0.36	1241.95	7252.99	0.63	1236.08	6570.08

Lasso	-0.36	1211.54	7264.04	0.63	1206.32	6582.28
ElasticNetCV	-0.13	1249.29	6625.63	0.62	1253.65	6652.26
Mean	0	2760.17	6230.05	0	2842.13	10745.52

3.3 Choosing model

Initial exploration of an untuned Random Forest for Gross Income (GI) prediction yielded poor results, and tuning was too time-consuming. Consequently, the research focused on XGBoost, a gradient boosting algorithm known for its efficiency and state-of-the-art performance (Chen & Guestrin, 2016). XGBoost's ability to handle categorical variables and leverage GPU acceleration for faster training made it a promising choice for achieving robust predictive accuracy for GI within the project's constraints. Therefore, XGBoost was selected as the primary modeling technique, with subsequent chapters detailing its implementation and evaluation.

3.4 XGBoost model

The XGBoost model inherently possesses a strong capability to handle categorical variables effectively, often outperforming approaches that rely on dummy encoding. Therefore, in this instance, I did not employ dummy encoding for the categorical features. Instead, I defined a reasonably broad hyperparameter grid and iteratively explored the impact of different parameter combinations to identify general trends. The tuning process indicated a preference for deeper trees, with the optimal `max_depth` identified as 23. Notably, a high L2 regularization term (`lambda`) of 50 was beneficial in controlling overfitting, while the L1 regularization term (`alpha`) was set to a much lower value of 0.2.

The primary metric used for model scoring and hyperparameter optimization was Mean Absolute Error (MAE). The resulting XGBoost model demonstrated a significant improvement over the previous baseline models, achieving an R-squared of 0.58 and an MAE of 961.31. While the model still exhibited some degree of overfitting when evaluating MAE on unseen data, this was deemed acceptable given the corresponding improvement in R-squared. Throughout the tuning process, a clear trade-off emerged: prioritizing a higher R-squared generally led to an increase in MAE, and conversely, minimizing MAE often resulted in a lower R-squared.

Table 3.2

Performance of XGBoost model on training and testing data (Author's calculation)

Model	R^2 Test	MAE Test	RMSE Test	R^2 Train	MAE Train	RMSE Train
XGBoost	0.58	961.31	4041.94	0.43	784.38	8123.73

Table 3.3

Parameter grid used for the final hyperparameter tuning of XGBoost, best hyperparameters are underlined (Author's calculation)

Hyperparameter	Values
N_estimators	225, <u>250</u> , 275
Max_depth	21, <u>23</u>
Learning_rate	0.01, <u>0.03</u> , 0.05
Subsample	0.2, <u>0.3</u> , 0.5
Colsample_bytree	0.7, <u>0.8</u>
Reg_alpha	0, <u>0.2</u> , 0.5
Reg_lambda	47, <u>50</u> , 53

3.5 Feature selection

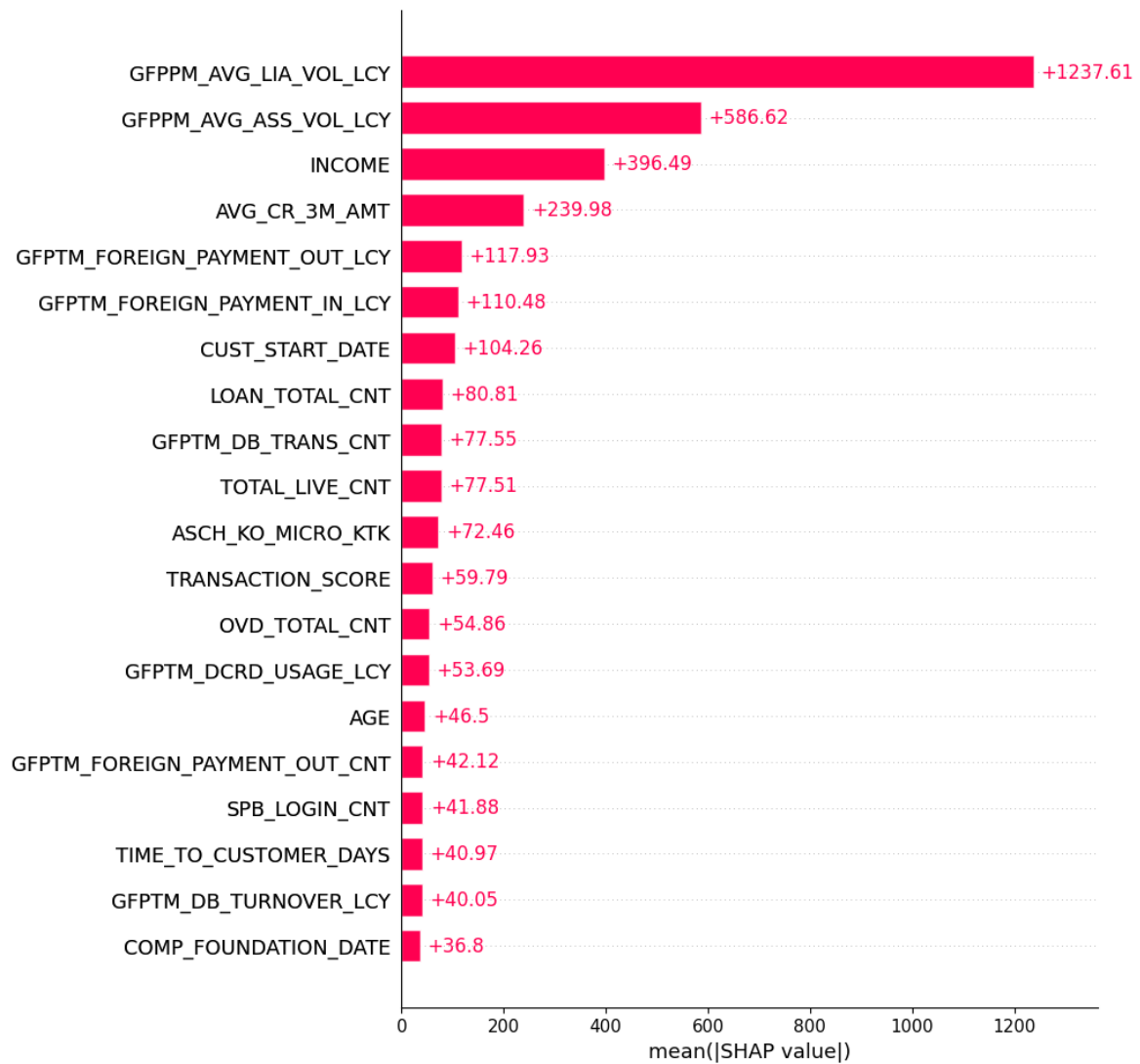
With a model now exhibiting relatively good predictive performance, the next step is to identify the most influential features impacting Gross Income (GI). For this purpose, I employed SHAP (SHapley Additive exPlanations) values. SHAP offers a distinct advantage over other feature importance methods by not only quantifying the importance of each feature but also illustrating its individual impact (positive or negative) on the target variable. Furthermore, SHAP is known to provide reliable feature importance estimates even in datasets with high autocorrelation, a characteristic present in my data.

My approach involves an iterative feature selection process. Initially, I identified the top 20 most important variables based on their mean absolute SHAP values. A new XGBoost model was then trained using only these 20 features to assess the impact of reducing the feature space. Subsequently, I repeated this process, selecting the top 15 features, then the top 10,

and finally the top 5, training a new model at each step. This iterative reduction in the number of features allows for an evaluation of the model's sensitivity to feature loss and can help determine a potentially optimal subset of features for prediction.

Figure 3.1

Mean SHAP value for top 20 features (Author's calculation)



4 Comparasion of models

The following graph and table illustrate that the performance difference between the models trained on varying numbers of top features is not substantial. Interestingly, the model utilizing the 15 most important features achieved the highest performance metrics. This outcome could be attributed to a particularly effective combination of hyperparameters identified during the tuning process for this specific feature subset. Beyond 15 features, a gradual decline in performance is observed, with the most significant drop occurring between the models trained on 10 and 5 features. This substantial decrease is likely due to the effective halving of the feature space, resulting in a considerable loss of potentially valuable information for the model.

Examining the table, the models demonstrate a relatively narrow range of performance variation across different feature subsets. The 'Full' model, utilizing all available features, achieved an R-squared of 0.58 and a Mean Absolute Error (MAE) of 961.31 on the test set.

Reducing the feature set to the top 20 features resulted in a marginal improvement in R-squared to 0.59, accompanied by a slight increase in MAE to 975.54 on the test set. Notably, the optimal performance on the test set, with an R-squared of 0.60 and a minimized MAE of 960.61, was attained by the model trained on the top 15 features. This suggests that a carefully curated subset of features can, in certain cases, outperform a model trained on the complete feature set.

A subsequent reduction in the feature count to 10 led to a decrease in R-squared to 0.57 and an increase in MAE to 982.61 on the test set. This trend continued with the model trained on the top 5 features, which exhibited an equivalent R-squared of 0.57 but the highest MAE of 1006.50 on the test set.

Analyzing the training metrics reveals a general decrease in R-squared and increase in MAE as the number of features diminishes, consistent with the expected reduction in model capacity. However, the superior performance of the 15-feature model on the test set, despite not exhibiting the highest training R-squared or lowest training MAE, underscores the importance of balancing model complexity and generalization. The substantial performance decline observed when reducing the feature set from 10 to 5 indicates that the excluded features contained information crucial for maintaining predictive accuracy.

Table 4.1

Performance of models on training and testing data (Author's calculation)

Model	R^2 Test	MAE Test	RMSE Test	R^2 Train	MAE Train	RMSE Train
Full	0.58	961.31	4041.94	0.43	784.38	8123.73

20 features	0.59	975.54	3980.94	0.48	866.83	7712.62
15 features	0.60	960.61	3953.56	0.47	815.45	7853.12
10 features	0.57	982.61	4081.80	0.32	984.40	8842.88
5 features	0.57	1006.50	4101.75	0.31	1030.40	8955.31

4.1 Feature comparsion

Examining the SHAP values for the top 10 features across the different models (Full, Top 20, Top 15, and Top 10), as visualized in the accompanying graph, reveals some interesting patterns. The graph allows for a direct comparison of feature importance as the number of features used in the model is reduced.

A notable observation is the consistency of the top 5 most important features across all four models. These features – GFPPM_AVG_LIA_VOL_LCY, GFPPM_AVG_ASS_VOL_LCY, INCOME, AVG_CR_3M_AMT, and GFPTM_FOREIGN_PAYMENT_OUT_LCY – maintain relatively similar mean absolute SHAP values, indicating their robust and persistent influence on the predicted Gross Income.

The lower five features within the top 10 exhibit more variability in their SHAP values and rankings across the different models. While some features, like CUST_START_DATE and GFPPM_FOREIGN_PAYMENT_IN_LCY, remain within the top 10 across most models, their relative importance fluctuates. Other features, such as LOAN_TOTAL_CNT, TOTAL_LIVE_CNT, and GFPTM_DB_TRANS_CNT, also appear in the top 10 but show some changes in their order and contribution depending on the feature subset used for training.

This relative stability in the top few features, coupled with the moderate changes in the lower-ranked ones, helps explain why the models did not experience a drastic drop in predictive performance despite the reduction in the total number of features. The core drivers of Gross Income appear to be consistently captured even with a smaller feature set, while the less impactful features contribute more marginally to the overall prediction.

Figure 4.1

Mean SHAP value for top 10 features for model Full, model Top 20, model Top 15 and model Top 10 (Author's calculation)

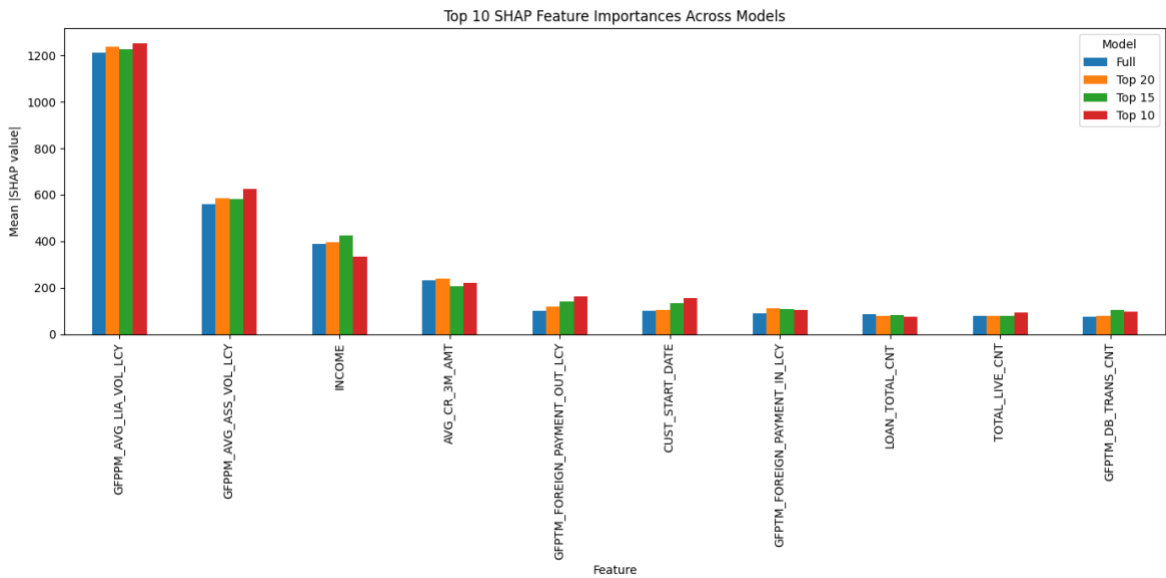
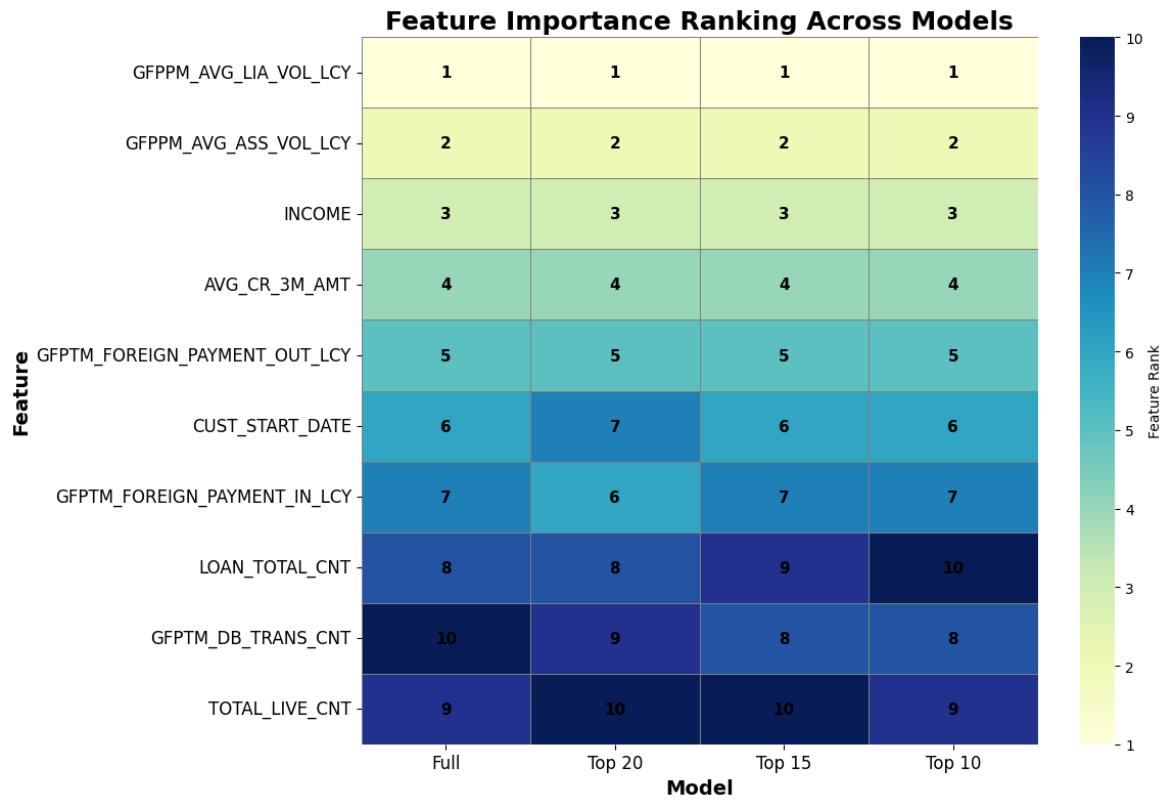


Figure 4.2

Ranking by SHAP value for top 10 features in model Full, model Top 20, model Top 15 and model Top 10 (Author's calculation)



4.2 Limitation of the results

Despite the relatively good predictive performance of the developed models, a significant portion of the variability in client profitability remains unexplained. Several factors could contribute to this unexplained variance, with the inherent nature of the data being a key consideration. Specifically, the target variable, Gross Income (GI), represents an average over a six-month period, while the majority of the explanatory variables capture client behavior within a single month. This temporal mismatch could lead to discrepancies. For instance, a client who recently paid off a substantial loan might exhibit low current asset and liability volumes, yet still reflect a high average GI from previous months when the loan was active. While such extreme scenarios are likely infrequent, this temporal disconnect introduces inherent limitations in capturing the full dynamics of profitability.

A second limitation stems from the scope of the available features. Although a considerable number of variables were included in the analysis, numerous other factors known to influence bank profitability are not directly captured. For example, the impact of various concurrent promotional campaigns can significantly affect client profitability by, for instance, temporarily waiving monthly account fees. Unfortunately, banking data can be inherently complex and 'messy,' with numerous unobserved or difficult-to-quantify factors potentially influencing GI. Pinpointing and incorporating all relevant variables presents a significant challenge in real-world banking data analysis. Furthermore, tree-based models like XGBoost inherently possess a degree of randomness in their training process, both in the selection of features at each split and the overall structure of the ensemble (Chen & Guestrin, 2016). While this randomness contributes to the model's robustness and ability to generalize, it also introduces a degree of variability in the learned relationships and feature importance scores across different training runs, potentially contributing to the unexplained variance.

5 Recommendation and insights

The SHAP summary plot ranks the features by their influence on the predicted Gross Income (GI). The most impactful features for predicting GI are situated at the top of the visualization. The feature exhibiting the greatest impact is GFPPM_AVG_LIA_VOL_LCY (Average Liability Volume LCY), where higher values strongly correlate with higher predicted GI, and lower values with lower predicted GI. This indicates that the scale of a client's liabilities is a key factor in predicting their income contribution.

Similarly, GFPPM_AVG_ASS_VOL_LCY (Average Asset Volume LCY) demonstrates a significant positive correlation with predicted GI, suggesting that clients with larger asset holdings tend to have higher predicted income contribution. The feature INCOME also plays a crucial role in predicting GI, with higher income levels clearly associated with higher predicted GI. AVG_CR_3M_AMT (Average Credit Amount Last 3 Months) indicates that clients who utilize more credit tend to have higher predicted GI. GFPPM_FOREIGN_PAYMENT_OUT_LCY (Outgoing Foreign Payments in CZK) doesn't seem to show a clear trend in predicting GI. GFPPM_FOREIGN_PAYMENT_IN_LCY (Incoming Foreign Payments LCY) shows a positive link to higher predicted GI, in some extreme cases even outperforming GFPPM_AVG_LIA_VOL_LCY, suggesting that clients involved in international payments are valuable contributors to predicted GI.

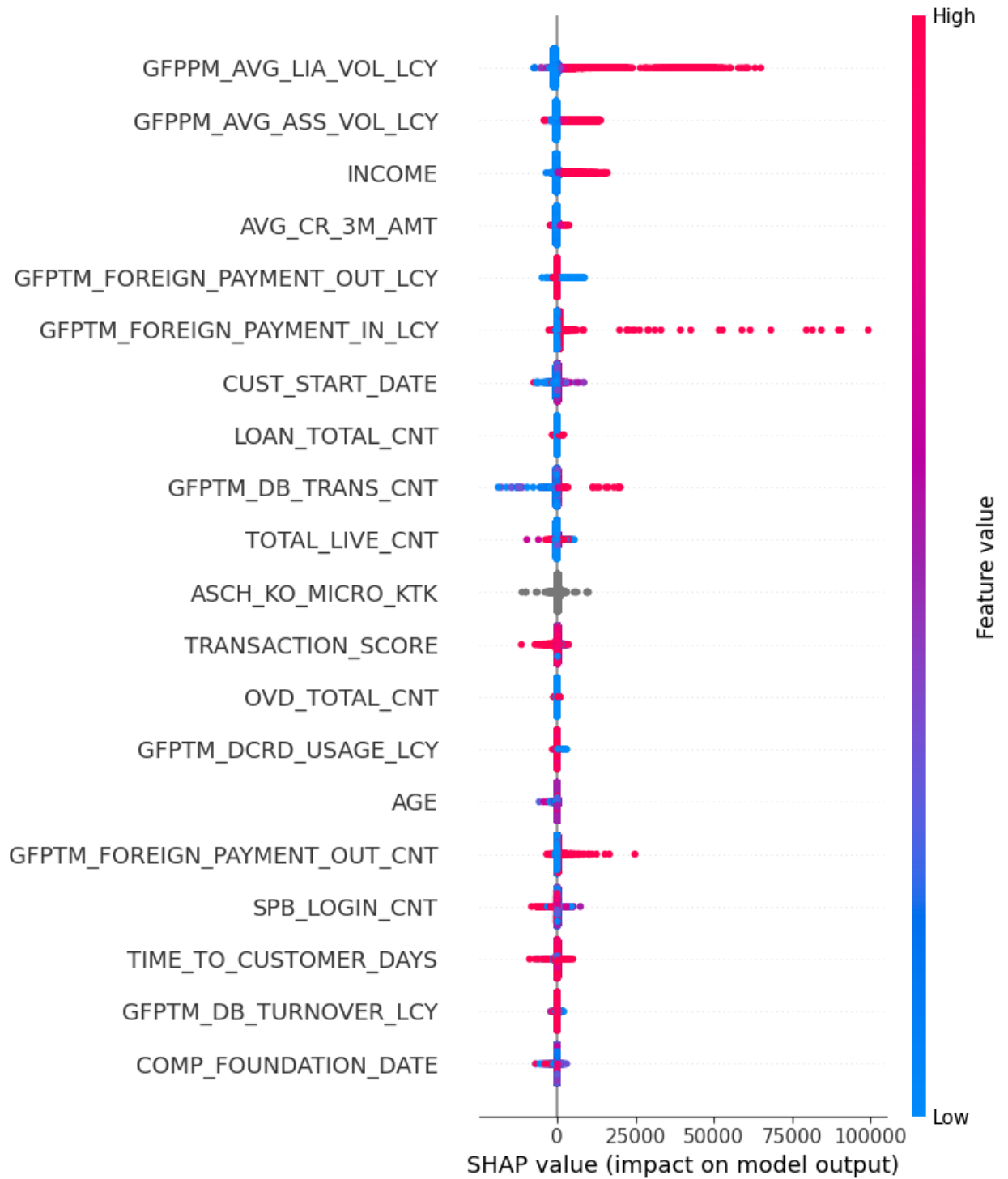
The impact of CUST_START_DATE (Customer Start Date) on predicted GI appears more complex, without a clear linear trend, suggesting a potentially nuanced relationship between client tenure and predicted GI. LOAN_TOTAL_CNT (Total Number of Loans) has a modest positive effect on predicted GI, indicating that clients with more loans tend to have slightly higher predicted GI. GFPTM_DB_TRANS_CNT (Debit Card Transaction Count) shows a weak positive association with predicted GI, suggesting that the frequency of debit card use is not a primary driver in predicting GI. The features lower on the plot have a less substantial impact on the model's predictions of GI. Interestingly, SPB_LOGIN_COUNT shows a negative effect on predicted Gross Income. While this correlation is observed, the underlying causality is unclear. For instance, larger, more profitable firms might simply log in to the app less frequently. However, other reasons could also be at play. Further investigation, potentially incorporating domain knowledge or specialized experiments, is needed to understand the causal factors behind why fewer logins might correlate with higher predicted profitability.

Based on these observations, Raiffeisenbank should prioritize strategies that focus on growing client liability and asset volumes, as these are strong indicators for predicting Gross Income. The positive correlation with income underscores the importance of attracting and retaining higher-earning clients for predicting GI. The value associated with international payment activity suggests a potential area for specialized services in predicting GI. Further investigation into the relationship between customer

tenure and predicted GI could yield valuable insights into long-term client value. Finally, while loan portfolio size shows a modest positive link to predicted GI, and debit card transaction frequency a weaker one, these factors could still be considered in a broader understanding of client engagement for predicting GI. In summary, the analysis highlights the importance of client financial volume, income, and international activity as key predictors of Gross Income for Raiffeisenbank.

Figure 5.1

Impact on model output for feature values of top 15 features



Conclusions

This thesis has directly addressed the core business objective: to identify and quantify the key financial and behavioral features that significantly predict Gross Income (GI) within Raiffeisenbank's micro and small enterprise client base. By developing and analyzing various machine learning models, particularly the high-performing XGBoost model, and employing insightful SHAP value analysis, this research has successfully identified a set of influential predictors of GI. Features such as average liability volume, average asset volume, client income, and international payment activity have consistently emerged as critical correlates of predicted GI, with their impact quantified through the SHAP values.

In response to the central research question – "What are the most important financial and behavioral characteristics of our clients that predict Gross Income, and how can we leverage this understanding to inform strategic decisions within the bank?" – this study provides concrete answers. The SHAP analysis demonstrates the direction and magnitude of these key characteristics' influence on predicted GI. Furthermore, the finding that a focused model utilizing the top 15 features achieves optimal predictive performance underscores the most salient indicators for understanding predicted GI.

The insights derived from this analysis offer actionable intelligence for strategic decision-making. Raiffeisenbank can leverage this understanding to prioritize client segments exhibiting these high-impact characteristics, tailor product offerings to enhance engagement in these areas, and refine marketing efforts to attract and retain clients with strong correlations to higher predicted GI. The evaluation framework, emphasizing predictive accuracy, feature stability and impact, and strategic relevance, has been addressed through the achieved model performance, the consistent identification of key features, and the formulated recommendations. While acknowledging the inherent complexities and limitations of real-world banking data, this research delivers a valuable, data-driven foundation for Raiffeisenbank to better understand its key predictors of profitability and optimize its strategic approach to maximizing predicted Gross Income and strengthening client relationships. The identified features serve as tangible levers for the bank to consider in its ongoing efforts to enhance financial performance.

For future research, several promising directions could be explored. One avenue would involve modifying the target variable to represent Gross Income for a single month rather than a six-month average. This, coupled with the enrichment of the dataset to include information on specific marketing campaigns, could provide a clearer understanding of their immediate impact on predicted profitability.

Another potential research direction would be to define distinct client profitability segments, such as unprofitable, profitable, and highly profitable clients. Subsequently, cluster analysis could be employed to identify the characteristics that differentiate these groups. This approach might enhance predictive accuracy and provide a more granular understanding of the factors correlated with profitability across different client segments.

List of references

- Rudin, C. (2019). Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1(5), 206-215. https://www.researchgate.net/publication/333069815_Stop_Explaining_Black_Box_Machine_Learning_Models_for_High_Stakes_Decisions_and_Use_Interpretable_Models_Instead
- Escalante, H. J., Escalera, S., Guyon, I., Baró, X., Güçlütürk, Y., Güçlü, U., & van Gerven, M. (Eds.). (2019). *Explainable and Interpretable Models in Computer Vision and Machine Learning* (The Springer Series on Challenges in Machine Learning). Springer Cham. <https://doi.org/10.1007/978-3-319-98131-4>
- Loh, W.-Y. (2011). Classification and regression trees. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, https://www.researchgate.net/publication/227658748_Classification_and_Regression_Trees
- Smith, C. (2018). *Decision Trees and Random Forests: A Visual Introduction For Beginners*. Blue Windmill Media. https://www.researchgate.net/publication/338389047_Decision_Trees_and_Random_Forests
- Burkov, A. (2019). *The Hundred-Page Machine Learning Book*. Andriy Burkov. <http://ema.cri-info.cm/wp-content/uploads/2019/07/2019BurkovTheHundred-pageMachineLearning.pdf>
- Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. In *Advances in neural information processing systems*¹(pp. 4765-4774). From : https://www.researchgate.net/publication/317062430_A_Unified_Approach_to_Interpreting_Model_Predictions
- Chen, T., & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. In KDD '16: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining <https://arxiv.org/abs/1603.02754>
- Ahmed, M. W. (2023). *Understanding Mean Absolute Error (MAE) in Regression: A Practical Guide*. From <https://medium.com/@m.waqar.ahmed/understanding-mean-absolute-error-mae-in-regression-a-practical-guide-26e80ebb97df>
- Jacobi, O. (2023, August 15). *Root Mean Square Error (RMSE): The cornerstone for evaluating regression models*. From <https://coralogix.com/ai-blog/root-mean-square-error-rmse-the-cornerstone-for-evaluating-regression-models/>
- Liu, X. (2023). *What Is R-Squared?* Investopedia. From <https://www.investopedia.com/terms/r/r-squared.asp>

XGBoost Documentation. (n.d.). *Parameters*. Retrieved May 6, 2025, from <https://xgboost.readthedocs.io/en/stable/parameter.html>

Bryanb. (n.d.). *XGBoost Explainability with SHAP*. Kaggle. Retrieved May 5, 2025, from <https://www.kaggle.com/code/bryanb/xgboost-explainability-with-shap>

Brownlee, J. (n.d.). *Feature Importance and Feature Selection with XGBoost in Python*. Machine Learning Mastery. Retrieved May 5, 2025, from <https://machinelearningmastery.com/feature-importance-and-feature-selection-with-xgboost-in-python/>

Wikipedia. (n.d.). *Black box*. Retrieved May 5, 2025, from https://en.wikipedia.org/wiki/Black_box

IBM. (n.d.). *What is a decision tree?* Retrieved May 5, 2025, from <https://www.ibm.com/think/topics/decision-trees>

VasquezAI, A. (n.d.). *Machine Learning*. GitHub. Retrieved May 5, 2025, from https://github.com/AlvaroVasquezAI/Machine_Learning

Awad, M., & Fraihat, S. (2023). Recursive Feature Elimination with Cross-Validation with Decision Tree: Feature Selection Method for Machine Learning-Based Intrusion Detection Systems. *Journal of Sensor and Actuator Networks*, 12(5), 67. <https://doi.org/10.3390/jsan12050067>

Berrar, D. (2018, January 1). Cross-Validation. <https://doi.org/10.1016/B978-0-12-809633-8.20349-X>

Bouchlaghem, Y., Akhiat, Y., & Amjad, S. (2022). Feature Selection: A Review and Comparative Study. *E3S Web of Conferences*, 351, 01046. <https://doi.org/10.1051/e3sconf/202235101046>

XGBoost Documentation. (2024, October 24). *Categorical Data*. Retrieved May 5, 2025, from <https://xgboost.readthedocs.io/en/stable/tutorials/categorical.html>

Lundberg, S. M., & Lee, S. I. (2020). Basic SHAP Interaction Value Example in XGBoost. *SHAP documentation*. Retrieved May 5, 2025, from https://shap.readthedocs.io/en/latest/example_notebooks/tabular_examples/tree_based_models/Basic%20SHAP%20Interaction%20Value%20Example%20in%20XGBoost.html

Datacamp. (n.d.). *Introduction to SHAP Values: Machine Learning Interpretability*. Retrieved May 5, 2025, from <https://www.datacamp.com/tutorial/introduction-to-shap-values-machine-learning-interpretability>

DataScientest. (n.d.). *SHapley Additive exPlanations or SHAP : What is it?* Retrieved May 5, 2025, from <https://datascientist.com/en/shap-what-is-it>

Goel, A. (2025, February 20). *Model exploitability using shap (shapley additive explanations) and lime (local interpretable)*. Medium. Retrieved May 5, 2025, from <https://medium.com/@anshulgoel991/model-exploitability-using-shap-shapley-additive-explanations-and-lime-local-interpretable-cb4f5594fc1a>

IBM. (n.d.). *What is lasso regression?* Retrieved May 5, 2025, from <https://www.ibm.com/think/topics/lasso-regression>

Corporate Finance Institute. (n.d.). *Elastic Net*. Retrieved May 5, 2025, from <https://corporatefinanceinstitute.com/resources/data-science/elastic-net/>

GeeksforGeeks. (2025, February 12). *Ridge Regression*. Retrieved May 5, 2025, from <https://www.geeksforgeeks.org/what-is-ridge-regression/>

Appendices

Appendix A: Source code

File *Bachelor_thesis_code.ipynb* contains code for all models and plots used in this thesis